

PromptShield: A Prompt-Injection Detection and Mitigation Gateway for LLM Applications

---

Project Proposal

Course: CS3002 Information Security | Fall 2026 (BSE-7A)

Group Members

Umer Naseer 23L-3040
Muhammad Arslan 23L-3025
Abubakar Afzal 23L-3062

National University of Computer and Emerging Sciences
Department of Computer Science
Lahore, Pakistan

Abstract

PromptShield is a lightweight security gateway for applications that use large language models (LLMs). It is designed to help identify prompt-injection attempts in user prompts and supported external content before they reach the model. The gateway combines practical rule-based checks with a TF-IDF and logistic-regression classifier, assigns a risk score, and applies a configurable action to allow, flag, or block a request. It also checks generated responses for selected leakage indicators. The project will be evaluated using a documented set of benign prompts and direct, indirect, and lightly obfuscated attack examples. PromptShield is a controlled academic prototype; it aims to reduce risk rather than guarantee detection of every attack.

1. Introduction

LLMs are increasingly used in chatbots, customer-support tools, email assistants, and document-processing applications. These systems often receive both trusted instructions and untrusted content, such as user messages, uploaded documents, or retrieved web pages. Prompt injection exploits this trust boundary by placing instructions in content that may cause a model to ignore its intended task, reveal protected information, or produce an unintended response.

Prompt injection can be direct, when a user includes malicious instructions in a prompt, or indirect, when instruction-like text is embedded in a document or other external content. Simple keyword filters may miss paraphrased or disguised attacks, while overly broad filters can incorrectly flag harmless requests. PromptShield will combine lightweight detection methods and transparent policy decisions to make the handling of suspicious inputs more consistent.

The gateway will sit between a small demo application and an LLM API. It will inspect supported inputs, record why a request was allowed, flagged, or blocked, and apply selected output checks. The project will not modify an LLM's weights or claim complete protection. Its contribution is a practical, testable demonstration of input validation, trust boundaries, defense in depth, and security logging.

2. Project Goal and Objectives

The main goal is to develop and evaluate a lightweight prompt-injection detection and mitigation gateway for a small LLM-powered application.

- Implement a rule-based scanner for common prompt-injection phrases and simple obfuscation patterns.
- Train and evaluate a TF-IDF plus logistic-regression classifier using a manageable collection of benign and attack prompts.
- Inspect text extracted from supported uploaded PDFs or supplied web content, treating that content as untrusted and flagging instruction-like patterns.
- Add output checks for a demo canary phrase and selected leakage indicators, including common API-key patterns and email addresses.
- Provide configurable risk thresholds and allow, flag, or block actions, with logs that record each decision and its reason.
- Build a small Python/FastAPI chatbot demo connected to an LLM API, with a switch to compare gateway-enabled and gateway-disabled behavior.
- Evaluate the prototype using precision, recall, F1-score, false-positive rate, attack success rate, and added processing latency.

3. Scope of the Project

3.1 In Scope

- Input inspection using rule-based checks and a TF-IDF/logistic-regression classifier.
- Basic checks for direct prompt injection and selected indirect-injection patterns in supported PDF text or supplied web text.
- An output guard that checks for a configured canary phrase and selected recognizable leakage patterns.
- A policy engine with configurable thresholds for allowing, flagging, or blocking requests.
- Decision logging for audit and analysis, including the selected action and its reason.
- A small Python/FastAPI demo application connected to an LLM API, with an enabled/disabled comparison toggle.
- A documented test set containing benign prompts and direct, indirect, and lightly obfuscated attack examples.

3.2 Out of Scope

- Training, fine-tuning, or changing the underlying LLM.
- Attacking or modifying model weights.
- Guaranteed detection or prevention of all prompt-injection techniques.
- Complete protection against every file format, hidden instruction, encoded attack, or multi-turn attack.
- Production-scale deployment or a claim that the prototype is ready to serve as a standalone commercial security product.

4. Importance and Applications

Prompt injection is an important security concern wherever an LLM processes user input or external content. A gateway provides a practical place to apply security controls before information reaches the model and to inspect responses before they are returned to the user. The project demonstrates several established security ideas: validation of inputs, separation of trusted instructions from untrusted data, defense in depth, and logging for review.

The prototype may serve as a learning model for customer-support chatbots, document assistants, and retrieval-based applications. Its value will be assessed in a controlled demonstration using the same test cases with the gateway enabled and disabled. Results will be reported with limitations, including false positives, missed attacks, and the possibility that attackers may use unfamiliar wording or encoding.

5. Methodology and Work Plan

- Review prompt-injection examples and collect a manageable public dataset containing both attack and benign prompts.
- Prepare and document a small test set, keeping training and test examples separate to reduce evaluation leakage.
- Implement a rule-based filter and measure its baseline performance on the test set.
- Train a TF-IDF plus logistic-regression classifier and compare its results with the rule-based baseline.
- Combine the scanner, output checks, policy settings, and decision logs in the demo application.
- Run controlled tests with the gateway enabled and disabled, measure the selected metrics, and document bypasses and limitations.
- Prepare setup instructions, architecture documentation, evaluation results, and a summary of each group member’s contribution.

6. System Design Overview

The proposed workflow begins when a user submits a prompt or supported external text to the demo application. PromptShield extracts or receives the text and passes it to the rule-based checks and the TF-IDF/logistic-regression classifier. Their outputs are combined into a risk score. The policy engine compares that score with configured thresholds and chooses to allow, flag, or block the request. If allowed, the request is sent to the connected LLM API. The generated response is then checked for the configured canary phrase and selected leakage patterns before being returned. The gateway records the decision and its reason for later inspection.

- Input layer: accepts user prompts and supported external text.
- Detection layer: combines rules and a lightweight text classifier.
- Policy layer: maps the risk score to allow, flag, or block actions.
- Output guard: checks generated text for selected demo leakage indicators.
- Logging and evaluation: records decisions and supports repeatable tests.

These checks are indicators rather than proof that content is safe or malicious. The system will expose its decisions for review and will document cases where the scanner misses an attack or incorrectly flags benign text.

7. Evaluation Metrics

- Precision: the proportion of prompts flagged as attacks that are attacks in the labelled test set.
- Recall: the proportion of labelled attack prompts correctly detected.
- F1-score: a combined measure of precision and recall.

- False-positive rate: the proportion of benign prompts incorrectly flagged or blocked.
- Attack success rate: comparison of outcomes for the same test cases with the gateway enabled and disabled.
- Added processing latency: average additional time introduced by the gateway per request.
- Robustness observations: performance on paraphrased, encoded, and multi-turn examples, with bypasses documented.

The evaluation will use a held-out test set that is kept separate from classifier training. Results will be reported for the rule-based baseline, the classifier, and the combined gateway where feasible. No performance result will be assumed in advance; measured outcomes will be included after implementation and testing.

8. Expected Deliverables

- Source code for the PromptShield gateway, rule-based baseline, classifier, and demo application.
- Setup and run instructions sufficient to reproduce the controlled demonstration.
- A short architecture document explaining the input scanner, policy engine, output checks, and logging.
- A documented test set and an evaluation report covering metrics, performance, limitations, and lessons learned.
- A contribution summary describing the work completed by each group member.

9. Requirements and Constraints

9.1 Functional Requirements

- The system shall accept prompts through the demo application and inspect them before sending permitted requests to the LLM API.
- The scanner shall apply rule-based checks and a trained TF-IDF/logistic-regression classifier.
- The policy engine shall support configurable allow, flag, and block outcomes.
- The output guard shall check for the selected canary phrase and supported leakage patterns.
- The system shall log the action taken and a human-readable reason.
- The demo shall allow a basic comparison between gateway-enabled and gateway-disabled runs.

9.2 Non-Functional Requirements and Constraints

- The prototype should remain lightweight enough to run in a student development environment.
- Tests should be repeatable, with documented prompts, labels, settings, and evaluation steps.
- Only supported PDF text extraction and supplied web text are promised; hidden or complex file content may not be fully inspected.
- Rule-based and pattern-based checks can produce false positives and false negatives.
- An LLM API connection may require an API key, network access, and compliance with the provider’s usage terms.

10. Risks and Limitations

Prompt injection is an evolving problem, and a small classifier trained on a limited dataset may not generalize to new attack styles. Rule-based checks can be bypassed by paraphrasing or encoding, while broad rules may flag harmless text. Output patterns such as email addresses or API-key-like strings are only indicators and do not establish that a real secret has been exposed. A canary phrase supports a controlled leakage demonstration but does not prove that all confidential information is protected.

To manage these limitations, the project will use multiple detection layers, document test cases, keep training and test data separate, and report both missed attacks and false alarms. The gateway will be presented as a risk-reduction prototype rather than a guarantee of security.

11. Conclusion

PromptShield proposes a practical gateway that combines rule-based inspection, lightweight text classification, configurable policy decisions, output checks, and logging around a small LLM application. The project will test whether these controls improve handling of a documented set of prompt-injection examples compared with running the same application without the gateway. The final report will present measured results and clearly state the limits of the approach.

References

[1] OWASP Foundation, “LLM Prompt Injection Prevention,” OWASP Gen AI Security Project. Available: https://genai.owasp.org/

[2] Scikit-learn developers, “TfidfVectorizer,” scikit-learn documentation. Available: https://scikit-learn.org/

[3] FastAPI, “FastAPI Documentation.” Available: https://fastapi.tiangolo.com/